Tuesday, January 13, 2026 12:14 AM

CloudHop Shell Theme Engine (Phase 2)

This engine gives you:

- global theme state
- theme tokens (colors, gradients, shadows)
- dynamic theme switching
- theme influence from subsystems (Spaces, Music Engine, etc.)
- future Leonardo-driven adaptive themes

Let's build it cleanly.

1. Theme Tokens (the design language)

These are the core visual identities CloudHop can shift between.

```
// shell/themeTokens.ts

export const themeTokens = {
  dark: {
    background: "linear-gradient(180deg, #0f172a, #1e293b)",
    panel: "rgba(15,23,42,0.9)",
    border: "rgba(148,163,184,0.2)",
    accent: "#38bdf8",
    text: "#e5e5e5",
  },

  neon: {
    background: "linear-gradient(180deg, #0f0f1f, #1a0033)",
    panel: "rgba(20,0,40,0.8)",
    border: "rgba(255,0,255,0.3)",
    accent: "#ff4dff",
    text: "#f0e6ff",
  },

  tactical: {
    background: "linear-gradient(180deg, #0a0f0f, #0f1f1f)",
    panel: "rgba(10,20,20,0.9)",
    border: "rgba(0,255,200,0.2)",
    accent: "#00e6b8",
    text: "#d9fff7",
  },

  ambient: {
    background: "linear-gradient(180deg, #0f172a, #334155)",
    panel: "rgba(30,41,59,0.8)",
    border: "rgba(148,163,184,0.3)",
    accent: "#93c5fd",
    text: "#f1f5f9",
  },
} as const;
```

These tokens give CloudHop a visual vocabulary.

2. Theme Engine Store (global state)
This extends your existing Shell state.

```
// shell/themeEngine.ts
import { create } from "zustand";
import { themeTokens } from "./themeTokens";

export type ThemeName = keyof typeof themeTokens;

interface ThemeEngineState {
  theme: ThemeName;
  tokens: typeof themeTokens[ThemeName];

  setTheme: (theme: ThemeName) => void;
  applyInfluence: (source: string, theme: ThemeName) => void;
}

export const useThemeEngine = create<ThemeEngineState>((set) => ({
  theme: "dark",
  tokens: themeTokens.dark,

  setTheme: (theme) =>
    set({
      theme,
      tokens: themeTokens[theme],
    }),

  applyInfluence: (source, theme) =>
    set({
      theme,
      tokens: themeTokens[theme],
    }),
}));
```

This gives you:

- direct theme switching
- subsystem-driven theme influence
- a single source of truth for all theme tokens

3. Integrating Theme Engine Into the Shell Layout
Modify your Shell root to use the theme tokens:

```
// shell/CloudHopShell.tsx

import { useThemeEngine } from "./themeEngine";

export const CloudHopShell = ({ runtime }) => {
  const { tokens } = useThemeEngine();

  return (
    <div
      style={{
        display: "grid",
        gridTemplateColumns: "70px 1fr",
```

```
      gridTemplateRows: "60px 1fr",
      height: "100vh",
      background: tokens.background,
      color: tokens.text,
    }}
  >
    {/* Top Bar */}
    <div style={{ gridColumn: "1 / span 2" }}>
      <ShellTopBar runtime={runtime} />
    </div>

    {/* Dock */}
    <ShellDock />

    {/* Main Surface */}
    <ShellSurface runtime={runtime} />
  </div>
 );
};
```

Now the entire Shell responds to theme changes instantly.

4. Theme Switcher UI (for Settings panel)

```
// shell/ThemeSwitcher.tsx

import React from "react";
import { useThemeEngine } from "./themeEngine";

export const ThemeSwitcher = () => {
 const { theme, setTheme } = useThemeEngine();

 const themes = ["dark", "neon", "tactical", "ambient"];

 return (
  <div style={{ display: "flex", gap: "12px" }}>
   {themes.map((t) => (
    <button
     key={t}
     onClick={() => setTheme(t)}
     style={{
      padding: "8px 12px",
      borderRadius: "6px",
      border: "1px solid rgba(148,163,184,0.4)",
      background: theme === t ? "rgba(255,255,255,0.1)" : "transparent",
      color: "#e5e5e5",
      cursor: "pointer",
      fontSize: "12px",
      letterSpacing: "0.08em",
      textTransform: "uppercase",
     }}
    >
     {t}
    </button>
```

```
    ))}
   </div>
  );
};
```

Drop this into your Settings app.

5. Subsystem Influence Hooks (optional but powerful)
This is where CloudHop becomes alive.
Music Engine → Ambient / Neon

```
engine.on("state", (s) => {
  if (s.soundtrackMode === "focus") {
    useThemeEngine.getState().applyInfluence("music", "ambient");
  }
  if (s.soundtrackMode === "party") {
    useThemeEngine.getState().applyInfluence("music", "neon");
  }
});
```

Spaces → Tactical / Ambient

```
space.on("mood:changed", (mood) => {
  if (mood === "calm") useThemeEngine.getState().setTheme("ambient");
  if (mood === "tech") useThemeEngine.getState().setTheme("tactical");
});
```

Leonardo (future) → Adaptive Themes
When he awakens, he can influence the Shell based on:

- user emotion
- Space energy
- music intensity
- time of day

But for now, he stays dormant.

CloudHop Shell Notifications System
This subsystem has three parts:

1. **Notification Store** — global state for toasts + history
2. **Toast Renderer** — transient pop-ups
3. **Notification Center** — a panel inside Settings or the Shell

Everything plugs into your existing Shell state and theme engine.
1. **Notification Store (global)**
This extends your Shell state with a clean API:

```
// shell/notifications.ts import { create } from "zustand"; export interface Notification { id: string;
message: string; time: number; type?: "info" | "success" | "warning" | "error"; } interface
NotificationState { toasts: Notification[]; history: Notification[]; push: (msg: string, type?:
Notification["type"]) => void; remove: (id: string) => void; } export const useNotifications = create((set)
=> ({ toasts: [], history: [], push: (message, type = "info") => { const n = { id: crypto.randomUUID(),
message, type, time: Date.now(), }; set((s) => ({ toasts: [...s.toasts, n], history: [...s.history, n], })); // auto-
remove toast after 4 seconds setTimeout(() => { set((s) => ({ toasts: s.toasts.filter((t) => t.id !==
n.id), })); }, 4000); }, remove: (id) => set((s) => ({ toasts: s.toasts.filter((t) => t.id !== id), })), }));
```

This gives you:

- push("message")
- auto-dismiss

- toast list
- history list

Everything you need.

2. **Toast Renderer (floating notifications)**

This sits at the root of the Shell and renders transient toasts.

```
// shell/NotificationToasts.tsx import React from "react"; import { useNotifications } from
"./notifications"; import { useThemeEngine } from "./themeEngine"; export const NotificationToasts = ()
=> { const { toasts, remove } = useNotifications(); const { tokens } = useThemeEngine(); return (
{toasts.map((t) => (
remove(t.id)} style={{ padding: "10px 14px", borderRadius: "8px", background: tokens.panel, border: `
1px solid ${tokens.border}`, color: tokens.text, fontSize: "13px", cursor: "pointer", boxShadow: "0 4px
12px rgba(0,0,0,0.3)", opacity: 0.95, transition: "opacity 0.2s ease", }} > {t.message}
))}
); };
```

This gives you:

- floating toasts
- theme-aware styling
- click-to-dismiss
- auto-dismiss

3. **Notification Center (history panel)**

This can live inside Settings or its own Shell app.

```
// shell/NotificationCenter.tsx import React from "react"; import { useNotifications } from
"./notifications"; import { useThemeEngine } from "./themeEngine"; export const NotificationCenter = ()
=> { const { history } = useNotifications(); const { tokens } = useThemeEngine(); return (
Notification Center
{history.length === 0 && (
No notifications yet
)} {history.map((n) => (
{new Date(n.time).toLocaleTimeString()}
{n.message}
))}
); };
```

This gives CloudHop a **system log**.

4. **Integrating Notifications Into the Shell**

Add the toast renderer to your Shell root:

```
// CloudHopShell.tsx import { NotificationToasts } from "./NotificationToasts"; export const
CloudHopShell = ({ runtime }) => { const { tokens } = useThemeEngine(); return (
); };
```

Now the Shell can speak.

5. **Subsystem Hooks (optional but powerful)**

Music Engine → Toasts

```
engine.on("state", (s) => { if (s.currentTrack) { useNotifications.getState().push(`Now playing:
${s.currentTrack.title}`); } });
```

Spaces → Toasts

```
space.on("user:join", (u) => { useNotifications.getState().push(`${u.name} joined the Space`); });
```

Twitch → Toasts

```
if (streamWentLive) { useNotifications.getState().push(`${channel} is now live`); }
```

Runtime → Toasts

```
runtime.on("session:start", (s) => { useNotifications.getState().push(`Game session started:
${s.gameName}`); });
```

CloudHop becomes **responsive**.

✨ What You Just Added

CloudHop now has:

- a global notification system
- floating toasts
- a notification center
- subsystem-driven alerts
- theme-aware styling
- a unified OS-level voice

This is a major step toward CloudHop feeling like a **living OS**.

Below is the full JSON structure, designed to mirror your TS API exactly while staying easy to load, patch, or override at runtime.

📄 StrikeCoreBranding.json (Complete Branding Schema)

```
{
  "header": {
    "title": "StrikeCore OS",
    "button": {
      "idle": "Initialize",
      "booted": "Operational"
    },
    "status": {
      "online": "Online",
      "idle": "Idle",
      "busy": "Busy",
      "in-game": "In Game",
      "in-space": "In Space",
      "listening": "Listening"
    }
  },

  "bootLabels": {
    "runtime:boot:network": "Initializing Network Stack",
    "runtime:boot:presence": "Syncing Presence Layer",
    "runtime:boot:telemetry": "Activating Telemetry",
    "runtime:boot:device": "Loading Device Profile",
    "runtime:boot:spaces": "Preparing Spaces Runtime",
    "runtime:boot:music": "Starting Music Engine",
    "runtime:boot:gamehub": "Mounting GameHub",
    "runtime:boot:complete": "Systems Online"
  },

  "panels": {
    "recommended": {
      "presence": "Presence & Activity",
      "network": "Network Operations",
      "telemetry": "Telemetry Monitor",
      "deviceProfile": "Device Profile",
      "deepLinkTester": "Deep Link Tester",
      "cloudSaves": "Cloud Saves",
```

```
    "lastSession": "Last Session",
    "eventLog": "Event Log"
  },
  "enhanced": {
    "presence": "StrikeCore Presence Layer",
    "network": "Network Diagnostics",
    "telemetry": "Telemetry & Metrics",
    "deviceProfile": "Device Identity",
    "deepLinkTester": "Deep Link Inspector",
    "cloudSaves": "Cloud Storage",
    "lastSession": "Session Archive",
    "eventLog": "System Event Log"
  },
  "minimalist": {
    "presence": "Presence",
    "network": "Network",
    "telemetry": "Telemetry",
    "deviceProfile": "Device",
    "deepLinkTester": "Links",
    "cloudSaves": "Saves",
    "lastSession": "History",
    "eventLog": "Events"
  }
 },

 "snippets": {
  "runtimePanelJSX": "<Panel title={StrikeCoreBranding.getPanelTitle('telemetry')}>...</Panel>",
  "badgeJSX": "<div className='sc-badge'>StrikeCore</div>",
  "watermarkJSX": "<div className='sc-watermark'>StrikeCore OS</div>",
  "sidebarJSX": "<Sidebar title={StrikeCoreBranding.header.title} />"
 }
}
```

How to Use This JSON in CloudHop

1. Load it at runtime

```
import branding from "./StrikeCoreBranding.json";
```

2. Replace your TS lookups with JSON lookups

```
branding.panels.recommended.network
branding.header.title
branding.bootLabels["runtime:boot:network"]
```

3. Keep your helper functions if you want

You can still wrap this JSON with:

```
export function getPanelTitle(key, variant = "recommended") {
  return branding.panels[variant][key] || key;
}
```

This keeps your API identical.

✨ Why this JSON version matters

- CloudHop Shell can theme itself dynamically
- Spaces can override branding per-Space
- GameHub can load tactical or neon variants
- Music Engine can influence panel naming
- Leonardo (when awakened) can rewrite labels contextually
- Branding becomes data, not code

This is the OS-grade way to do it.

Localized Variants (Complete Set)

You can merge these into your JSON or TS module under:
branding.localized.short
branding.localized.ultra
branding.localized.hud

▨ Short Variant (clean, compact)

```
{
  "presence": "Presence",
  "network": "Network Ops",
  "telemetry": "Telemetry",
  "deviceProfile": "Device Profile",
  "deepLinkTester": "Link Tester",
  "cloudSaves": "Cloud Saves",
  "lastSession": "Last Session",
  "eventLog": "Event Log",

  "boot:network": "Network Init",
  "boot:presence": "Presence Sync",
  "boot:telemetry": "Telemetry Init",
  "boot:device": "Device Load",
  "boot:spaces": "Spaces Prep",
  "boot:music": "Music Engine",
  "boot:gamehub": "GameHub Load",
  "boot:complete": "Online"
}
```

▨ Ultra-Short Variant (tactical abbreviations)

```
{
  "presence": "PRS",
  "network": "NET",
  "telemetry": "TEL",
  "deviceProfile": "DEV",
  "deepLinkTester": "DLT",
  "cloudSaves": "CLS",
  "lastSession": "LST",
  "eventLog": "LOG",

  "boot:network": "NET↑",
  "boot:presence": "PRS↑",
  "boot:telemetry": "TEL↑",
```

```
  "boot:device": "DEV↑",
  "boot:spaces": "SPC↑",
  "boot:music": "MSC↑",
  "boot:gamehub": "GH↑",
  "boot:complete": "OK"
}
```

These are perfect for:

- sidebar icons
- micro-panels
- HUD overlays
- diagnostics mode

▨ HUD Variant (stylized, high-signal)

```
{
  "presence": "[PRESENCE]",
  "network": "[NETWORK]",
  "telemetry": "[TELEMETRY]",
  "deviceProfile": "[DEVICE]",
  "deepLinkTester": "[DEEPLINK]",
  "cloudSaves": "[CLOUD]",
  "lastSession": "[SESSION]",
  "eventLog": "[EVENTS]",

  "boot:network": ">> NET: ONLINE",
  "boot:presence": ">> PRESENCE: SYNC",
  "boot:telemetry": ">> TELEMETRY: ACTIVE",
  "boot:device": ">> DEVICE: READY",
  "boot:spaces": ">> SPACES: PREP",
  "boot:music": ">> MUSIC: READY",
  "boot:gamehub": ">> GAMEHUB: MOUNTED",
  "boot:complete": ">> SYSTEMS ONLINE"
}
```

These are ideal for:

- StrikeCore diagnostics
- boot sequence overlays
- GameHub tactical mode
- Spaces “tech” mood themes

🌀 How to integrate these variants

Add them to your branding module:

```
localized: {
  short: { ... },
  ultra: { ... },
  hud: { ... }
}
```

Then expose a helper:

```
getLocalized(key: string, variant: "short" | "ultra" | "hud") {
  return StrikeCoreBranding.localized[variant][key] || key;
}
```

Now you can do:

```
getLlocalized(key: string, variant: "short" | "ultra" | "hud") {
  return StrikeCoreBranding.localized[variant][key] || key;
}
```

or

```
<Panel title={StrikeCoreBranding.getLocalized("network", "short")} />

<BootLine>{StrikeCoreBranding.getLocalized("boot:network", "hud")}</BootLine>
```

emoji-accented variant set.
These are perfect for:

- CloudHop Shell sidebars
- Spaces mood-aware panels
- Music Engine overlays
- GameHub mini-HUDs
- Boot sequence flavor text
- Compact UI where a little personality goes a long way

I'll keep them tasteful, OS-grade, and aligned with CloudHop's emotional tone — expressive but not childish.
Below are three emoji-accented styles, all mapped to your existing keys:

✨ Emoji Variant Set (Full, Drop-In Ready)
You can add these under:
branding.localized.emoji

🎨 1. Soft-Emoji Variant (friendly, readable)

```
{
  "presence": "Presence 🌐",
  "network": "Network Ops 📡",
  "telemetry": "Telemetry 📊",
  "deviceProfile": "Device Profile 💻",
  "deepLinkTester": "Link Tester 🔗",
  "cloudSaves": "Cloud Saves ☁",
  "lastSession": "Last Session 🕒",
  "eventLog": "Event Log 📜",

  "boot:network": "Initializing Network 📡",
  "boot:presence": "Syncing Presence 🌐",
  "boot:telemetry": "Activating Telemetry 📊",
  "boot:device": "Loading Device Profile 💻",
  "boot:spaces": "Preparing Spaces 🌀",
  "boot:music": "Starting Music Engine 🎶",
  "boot:gamehub": "Mounting GameHub 🎮",
  "boot:complete": "Systems Online ⚡"
}
```

This variant is expressive but still clean enough for everyday UI.

⚡ 2. Tactical-Emoji Variant (sharp, high-signal)

```
{
  "presence": "Presence 🧩",
  "network": "Network ⚡",
  "telemetry": "Telemetry 🧭",
  "deviceProfile": "Device 🖥",
  "deepLinkTester": "DeepLink 🧩",
  "cloudSaves": "Cloud ☁",
  "lastSession": "Session 📂",
  "eventLog": "Events 📋",

  "boot:network": "NET ↑ ⚡",
  "boot:presence": "PRESENCE ↑ 🧩",
  "boot:telemetry": "TELEMETRY ↑ 🧭",
  "boot:device": "DEVICE ↑ 🖥",
  "boot:spaces": "SPACES ↑ 🌀",
  "boot:music": "MUSIC ↑ 🎶",
  "boot:gamehub": "GAMEHUB ↑ 🎮",
  "boot:complete": "ONLINE ⚡"
}
```

This one feels like a StrikeCore tactical HUD — perfect for diagnostics or GameHub.

🌈 3. Mood-Emoji Variant (CloudHop-style emotional resonance)
This one is tuned to CloudHop's vibe — expressive, atmospheric, alive.

```
{
  "presence": "Presence ✨",
  "network": "Network 🌐",
  "telemetry": "Telemetry 🔭",
  "deviceProfile": "Device 🌙",
  "deepLinkTester": "DeepLink 🎯",
  "cloudSaves": "Cloud Saves ☁",
  "lastSession": "Last Session 🪐",
  "eventLog": "Event Log 📜",

  "boot:network": "Network Warming 🌐",
  "boot:presence": "Presence Syncing ✨",
  "boot:telemetry": "Telemetry Awakening 🔭",
  "boot:device": "Device Profile Loading 🌙",
  "boot:spaces": "Spaces Coming Online 🪐",
  "boot:music": "Music Engine Resonating 🎶",
  "boot:gamehub": "GameHub Activating 🎮",
  "boot:complete": "Systems Alive ✨"
}
```

This variant is perfect for:

- Spaces with emotional moods
- Music Engine reactive themes
- CloudHop Shell ambient mode

How to integrate

Add to your branding module:

```
localized: {
  emoji: {
    soft: { ... },
    tactical: { ... },
    mood: { ... }
  }
}
```

Helper:

```
getEmoji(key: string, variant: "soft" | "tactical" | "mood") {
  return StrikeCoreBranding.localized.emoji[variant][key] || key;
}
```

Usage:

```
<Panel title={StrikeCoreBranding.getEmoji("network", "tactical")} />
```

or in boot:

```
<BootLine>{StrikeCoreBranding.getEmoji("boot:spaces", "mood")}</BootLine>
```

If you want, I can now generate: